

Full Codebase Report

A complete, file-by-file read of every subsystem in the ReeferReady PMS monorepo — architecture, the API, the database, the shared domain package, the web frontend, the design-system generation pipeline, the test suites, and workspace tooling — together with every genuine bug, inconsistency and risk the read surfaced.

Method: 10 parallel subagents, one per subsystem, each reading every listed file in full (no skimming/grepping), followed by a cross-cutting synthesis pass. 175 tool calls total across the read.

Scope: packages/shared, apps/api (routes, auth, Prisma schema, seed data, tests), apps/web (lib, components, screens), the Crystal design-system extractor and its source/generated output, the Playwright e2e suite, and root/workspace tooling.

Generated: 2026-09-23

Contents

1. What This Project Is	3
2. Architecture — A Real Request Traced End to End	3
3. packages/shared — The Domain Core	4
Findings	4
4. apps/api — The Backend	5
4.1 Core plumbing	5
4.2 Routes and integration tests	6
4.3 Database schema and seed data	7
5. apps/web — The Frontend	8
5.1 Core and lib (the composition root)	8
5.2 Components — app-level and the Crystal design-system layer	8
5.3 Screens	9
6. The Design-System Pipeline	10
Two genuine, previously-undetected bugs — found by comparing source HTML against generated output . .	10
7. End-to-End Test Suite (Playwright)	11
8. Tooling and Workspace Config	11
9. Auth/Permissions Model (Cross-Cutting)	12
10. Consolidated Cross-Cutting Risks, Ranked	13
11. Overall Code-Quality Assessment	14
Strong	14
Weak	14

1. What This Project Is

ReeferReady PMS is a container readiness pipeline for a reefer depot network: ten repair stages from gate-in to release, a checklist that varies by container type, and one view of what is delayed across four depots. Four roles are modeled — manager, supervisor, technician, viewer — and the whole visual layer is built on a hand-authored, code-generated “Crystal” design system.

Stack: an npm-workspaces monorepo with no Turborepo/Nx — orchestration is plain `npm -w` flags plus `concurrently`.

File	Role
packages/shared	Dependency-free domain core (readiness rules + zod schemas), consumed as raw TypeScript by both sides.
apps/api	Fastify 5 + Prisma 6 + SQLite backend, JWT auth, audit trail.
apps/web	React 19 + Vite frontend: TanStack Query/Table, Radix UI, cmdk, TipTap, bound to the generated Crystal CSS/icons.
tools/extract-design-system.mjs	The code-generation bridge from the design-system source HTML into the app's CSS/icons.

The repo's stated organizing principle (README) is that **“no value is ever typed twice.”** That claim holds completely for design tokens and mostly holds for domain logic — but this read found several concrete places where it is violated in practice (see §9).

2. Architecture — A Real Request Traced End to End

A user ticks a checklist item in `ContainerDrawer.tsx`:

- `useToggleTask` (`lib/queries.ts` L56–63) fires `api.toggleTask(id, key, done)`.
- `lib/api.ts`'s `request()` (L66–96) attaches the bearer token, POSTs `/api/containers/:id/tasks`, and converts any non-2xx response into a typed `RequestError` (status/code/fields/payload).
- Server-side, `routes/containers.ts` runs the request through `authenticate` (a global preHandler hook, L27) then `requirePermission('container:task')` (both from `auth.ts`).
- The handler (L269–307) runs a `prisma.$transaction(...)` that flips the task's `done/doneAt/doneBy` and inserts a `ContainerEvent` audit row atomically — the audit line can never desync from the mutation it describes.
- The container is re-fetched via `containerInclude + toContainer()` (`mappers.ts`) and returned with the server's `today` (`today.ts`).
- The client patches/invalidates the TanStack Query cache; every screen watching `useOverview()/useContainer()` re-renders.

The load-bearing decision: `packages/shared` is not just shared types — it is the only place business rules are *computed*. `containers.ts` imports `isLate`, `isReady`, `nextStage`, `checkListFor`, `stageBudget` directly rather than reimplementing them, and even filters “late/ready” in application code instead of SQL specifically so there is exactly one definition. The web side calls the identical functions to render status pills, progress and the work queue. This is what makes “API and UI can never disagree” structurally true rather than aspirational.

3. packages/shared — The Domain Core

File	Role
index.ts	4-line barrel — <code>export *</code> from the three modules below. Zero logic.
process.ts	Static business data: 10 fixed stages + day budgets, the ~48–51-item task-template library per container type (<code>checklistFor</code>), variant-extra-day rules (<code>extraDaysForStage</code>), plan totals. No imports — the leaf of the dependency graph.
readiness.ts	Pure derivations: progress %, <code>isLate/isReady</code> , the <code>Status</code> (tone+icon+label+detail) object that structurally enforces “never colour alone,” the prioritized work queue (<code>tomorrowTasks</code>), and greedy order-to-inventory matching (<code>matchOrders</code>). <code>today</code> is always passed in, never read from the clock.
readiness.test.ts	Behavior-pinning Vitest suite (“how it looks changes, what it does does not”) — no mocks, exercises real <code>STAGES/TASK_TEMPLATES</code> data.
schemas.ts	zod contracts built from <code>process.ts</code> ’s own enums, so API validation and web form types cannot drift apart.

Findings

- `<b>containerId` schema does not validate what its own comment says it does (`schemas.ts` L26–31): the comment describes real ISO 6346 format (4 letters + 7 digits) and claims it “normalises rather than rejects” — but the actual rule is only trim + 4–20 char length bound + whitespace collapse. No regex, no check-digit. `"AAAA"` or `"1234"` would pass straight through.
- Calendar dates are never actually validated — `isoDate` (`schemas.ts` L17–19) and `parseISODate` (`readiness.ts` L99–102) only check the `YYYY-MM-DD` shape. `2026-02-30` silently rolls over to March via native `Date` normalization instead of being rejected anywhere in the shared package.
- `totalPlanDaysFor` in `readiness.ts` (L133–135) reimplements `totalPlanDays` from `process.ts` instead of calling it — duplication inside the one package whose entire job is to prevent duplication.
- `<b>Latent key-collision risk</b>`: `TASK_TEMPLATES` keys (e.g. `mechanical.07`) are positional, generated from array index (`process.ts` L231–239), documented as “stable” — true only for in-place renames. Inserting/reordering a task mid-array would silently reassign every later task’s key in that stage. Since `averageHoursFor()` aggregates fleet-wide by key, old and new containers could end up sharing a key that means two different things, silently corrupting the average. Nothing enforces “append-only,” no test guards it.

4. apps/api — The Backend

4.1 Core plumbing

File	Role
app.ts	Composition root: registers @fastify/sensible/cors/rate-limit/jwt, a global error handler (Zod→422, 4xx pass-through, 5xx always generic — never leaks internals), a uniform 404, /api/health, and the four route plugins. ignoreTrailingSlash: true specifically because container IDs contain spaces that arrive percent-encoded.
auth.ts	Security core: script password hashing (timingSafeEqual comparison, no native deps), the ROLES/PERMISSIONS table (flagged in its own comment as “NEEDS SIGN-OFF”), can(), and the two Fastify preHandlers authenticate/requirePermission.
db.ts	Single shared PrismaClient singleton.
env.ts	zod-validates all env vars at import time, fails fast with a readable error. Deliberately re-applies pre-existing process.env values after loading .env, so the test runner’s forced DATABASE_URL can never be silently overridden by a stray .env file.
http.ts	zodToApiError, parseOr422, notFound/conflict — the one error-shape vocabulary every route uses.
mappers.ts	Translates Prisma rows → shared DTOs (toContainer/toOffLease/toOrder); atLocalMidnight() avoids the classic UTC-shift date bug.
server.ts	Entry point — builds the app, listens on 0.0.0.0:PORT, graceful shutdown on SIGINT/SIGTERM.
today.ts	Resolves “today” per-request; honors a client-supplied x-today header outside production only — the deterministic-time seam the whole test suite depends on.

- .env.example’s placeholder JWT_SECRET is long enough to pass validation (44 chars vs. a 16-char minimum) — a deployment that copies the example to .env without changing it, as instructed, boots successfully on a publicly-known signing secret.
- mappers.ts does JSON.parse(t.onlyFor) with no try/catch — a malformed onlyFor column would 500 the entire container-fetch.
- mappers.ts casts DB strings straight to shared union types (ContainerType, Priority, StageId, ...) via plain as — no runtime check.
- today.ts’s escape hatch is gated on NODE_ENV !== 'production', not specifically test — any non-prod deployment (e.g. a staging box mistakenly run as development) would accept a caller-spoofed “today.”
- No process-level unhandledRejection/uncaughtException handler in server.ts — an out-of-request-lifecycle error skips the graceful-shutdown/Prisma-disconnect path.

4.2 Routes and integration tests

File	Role
routes/auth.ts	POST /api/auth/login (10/min rate-limited, anti credential-stuffing) and GET /api/auth/me.
routes/containers.ts	The workhorse — list/search, read-one-with-audit-trail, register (snapshots the checklist so later template edits never rewrite completed work), edit, tick-task (transactional), advance-stage (force-past-open-tasks contract), delete, checklist-library.
routes/overview.ts	One big server-computed aggregate feeding Dashboard + Depot Command.
routes/reference.ts	Read-only form metadata (depots, active technicians, vocab).
__tests__/api.test.ts	The whole integration suite — real seeded SQLite via <code>.inject()</code> , no mocks.
__tests__/globalSetup.ts	Pushes schema + reseeds test.db once before the run.
vitest.config.ts	<code>fileParallelism: false</code> (shared SQLite file), generous hook timeout for the push+seed.

Real bugs found

- **Login's anti-enumeration is broken by a timing side-channel.** `routes/auth.ts` L21: `const ok = user ? verifyPassword(...) : false` — the deliberately-slow bcrypt verification only runs when a matching user row exists. A non-existent email short-circuits instantly with no hashing, so response *latency* leaks whether an email is registered — directly contradicting the route's own “roughly the same work either way” comment. `api.test.ts`'s enumeration test (L43–57) only checks the response message, not timing, so this is untested.
- **Deleting a container deletes its own audit trail.** DELETE /api/containers/:id cascades `ContainerEvent` rows away with the container (confirmed by schema + test at `api.test.ts` L317–333). The only surviving record is a `request.log.warn` line — not a queryable record. For a product whose headline feature is an audit trail, deletion is the one action it doesn't actually retain.
- **overview.ts's stated rationale doesn't match its payload** — the docstring justifies server-side aggregation as “the phone doesn't download the whole fleet,” but the response still embeds the complete `containers` array with full checklists, so the client downloads full detail twice (once from /overview, again from /containers).
- `overview.ts/reference.ts` do unpaginated, unfiltered `findMany` across every table on every request, with no depot-scoping despite `SessionUser.depotId` existing on the JWT.
- The `homeDepot` fallback expression is duplicated verbatim between `overview.ts` and `reference.ts`.
- **api.test.ts never signs in or tests the supervisor role at all** — only manager, technician, viewer. Supervisor (create/update/advance/task, not delete) has zero test coverage.
- The “working a container”/“permissions” test blocks are stateful and order-dependent, mutating the same container across `it` blocks rather than isolated fixtures.

4.3 Database schema and seed data

File	Role
<code>prisma/schema.prisma</code>	Eight models: User, Depot, Technician, Container (PK = the real ISO 6346 unit number, no @default), ContainerTask (snapshotted per-container), ContainerEvent (the audit trail), OffLeaseUnit, CustomerOrder. Thoughtful onDelete variety (Restrict while a depot/technician holds containers, Cascade for a container's own children, SetNull for soft un-links).
<code>prisma/seed.ts</code>	Deliberately hand-authored, deterministic fixture (not random) — 4 depots, 8 technicians in 4 buddy pairs, 4 users (one per role), 10 containers positioned to exercise every dashboard state, 5 off-lease units, 5 orders.

- **No native Prisma enums anywhere** — role, container type, stage, priority, event kind, grade are all plain String columns; validity is a comment-only convention, not a DB constraint.
- **“Append-only” audit trail is a documentation convention only** — nothing in the schema (trigger, restricted grant, immutability) actually prevents an UPDATE/DELETE on `ContainerEvent`.
- **Audit fields are free text, not FKs** — `ContainerTask.doneBy` and `ContainerEvent.actor` are plain strings, not references to User/Technician.
- `Depot.isHome`’s “exactly one home depot” invariant is documented, not enforced (schema permits zero or many).
- **No prisma/migrations directory exists** — the workflow is `db push`, not versioned migrations, which undercuts the schema header’s own claim that moving to Postgres later is “a provider swap plus a migration.”
- `seed.ts` hardcodes its own `STAGE_ORDER` array instead of importing STAGES from `packages/shared` — a literal duplication of the one thing the project says is defined exactly once.
- `User` has no relation to `Technician` — the “same person” link is a naming coincidence in seed data only.
- `seed.ts` hardcodes and prints a shared dev password (`readiness`) to stdout.

5. apps/web — The Frontend

5.1 Core and lib (the composition root)

File	Role
App.tsx	Top-level route table + auth gate; hardcodes 8 route paths independently of lib/nav.ts.
main.tsx	Provider stack: StrictMode → ThemeProvider → QueryClientProvider → AuthProvider → Tooltip → Toast → BrowserRouter → App. Thoughtful retry policy (never retry 4xx, retry transport failures up to 2×).
lib/api.ts	The one HTTP client; typed RequestError; localStorage token store with graceful in-memory fallback.
lib/auth.tsx	Session context; client-side PERMISSIONS table explicitly commented as UX-only mirror of the server.
lib/queries.ts	TanStack Query hook layer over every api.* call; every mutation invalidates overview.
lib/format.ts	Centralized date/duration formatting.
lib/nav.ts	The one declarative nav structure (sidebar/palette/headers all read it).
lib/theme.tsx / lib/cx.ts	Light/dark theme context; trivial classnames helper.
vite.config.ts / index.html	Dev-server API proxy; pre-paint theme bootstrap script (mirrors theme.tsx by necessity).

- `formatDateTime` violates its own module's stated invariant. `format.ts`'s doc-comment says dates must render identically regardless of viewer timezone; `formatDate/formatDateShort` honor that, but `formatDateTime` (L25–30) uses `new Date(iso) + .getHours()/.getMinutes()` — both browser-local — so an audit-trail event's displayed time differs between two users in different timezones viewing the same event.
- App.tsx's route table and lib/nav.ts's nav config independently hardcode the same 8 paths — currently in sync, nothing enforces it stays that way.
- App.tsx's NotFound uses a plain `<a href>` instead of `<Link>`, forcing a full page reload.
- No mutation hook in lib/queries.ts defines an onError handler at the hook level — error surfacing is left entirely to call sites.

5.2 Components — app-level and the Crystal design-system layer

App-level (`components/app`): `AddContainerDialog.tsx` (registration form, validated against the same zod schema the API uses), `AppShell.tsx` (sidebar rail/nav sheet/app bar, mounts the two global overlays), `CommandPalette.tsx` (■K, cmdk-based), `ContainerDrawer.tsx` (the heaviest integration file — checklist, advance, notes, history, removal, all opened via a `?container=<id>` URL contract), `ContainerGauge.tsx` (readiness drawn as the container itself, stage ribs weighted by task count), `KanbanBoard.tsx` (the ten-stage board).

Crystal design-system primitives ([components/crystal](#)): [Button.tsx](#), [Data.tsx](#) (status pills structurally forced to carry tone+icon+label — “rule 12” made structural, not just conventional), [DataTable.tsx](#) (TanStack Table + a responsive cards fallback driven entirely by column metadata), [Feedback.tsx](#) (alerts/toasts/AsyncRegion), [Form.tsx](#), [Icon.tsx](#) (the single icon primitive, backed by the generated sprite), [Overlay.tsx](#) (Radix-backed modal/drawer/menu/confirm-dialog), [RichText.tsx](#) (TipTap-based handover-note editor, explicitly not `execCommand`, per a documented design-system exception).

Two real bugs found here

- Overlay.tsx L39 — dead/no-op ternary: `aria-describedby={description ? undefined : undefined}`. Both branches evaluate to `undefined`. Harmless today because no caller passes `description`, but it is clearly unfinished code (most plausibly meant to be `description ? descId : undefined`), and if a future caller did pass `description`, this would silently suppress Radix’s own automatic `aria-describedby` wiring.
- RichText.tsx L179–182 — premature “saved” state: the Save button calls `setDirty(false)` immediately after invoking `onSave(...)`, without waiting for the async mutation to actually succeed. If the save fails, `ContainerDrawer` shows an error toast, but the editor’s own “unsaved” indicator and Save button have already cleared — the UI silently reads as saved even though nothing persisted.

Other findings: `AddContainerDialog` always defaults the new technician assignment to `technicians[0]` (biases every registration toward one person); `Data.tsx`’s `CategoryBadge` internal class names read like diff/changelog vocabulary (`t-new/t-existing`) rather than its own category vocabulary; `DataTable.tsx` opens a row via the first visible cell, not necessarily the column flagged `lead` in its own metadata contract; `AsyncRegion` renders raw `error.message` directly to end users of any role.

5.3 Screens

Nine route-level components ([screens/](#)), all thin — none fetch ad hoc or own business logic; seven of nine call the single `useOverview()` hook and derive everything from `@pms/shared`.

File	Role
Dashboard.tsx	The app’s one hero card (design-system rule 4) + 4 stat cards + the shared Kanban board.
Pipeline.tsx	The Kanban board alone, no hero.
DepotCommand.tsx	Landing screen (<code>/</code> → <code>/depot</code>): home-fleet, network summary, off-lease inbound, order-to-inventory matching.
Fleet.tsx	The full searchable/filterable/sortable container register.
Delayed.tsx	Every container past its stage budget, worst-offenders-first.
Tomorrow.tsx	The technician work queue — every outstanding task, flattened, with backup coverage and fleet-average hour comparisons.
Timeline.tsx	Static planned-duration bars per stage — deliberately never uses status colour.
ChecklistLibrary.tsx	Read-only browse of the master task template library.
Login.tsx	Sign-in screen, validated against the shared loginSchema.

- Login.tsx L157–168 hardcodes plaintext demo credentials in the rendered UI — three of four roles, all sharing the password `readiness`. No supervisor account is shown anywhere.

- Dashboard and Pipeline render the identical `<KanbanBoard>`; Delayed/Fleet/Tomorrow each independently redefine near-identical TanStack column boilerplate — real, if mild, duplication.
- Tomorrow.tsx recomputes `averageHoursFor()` once per row inside a single useMemo — $O(\text{rows} \times \text{containers})$, harmless at demo scale.
- Fleet.tsx's "clear filters" handler rebuilds URL search params for a case that can't currently occur (search/filter were never written to the URL).
- DepotCommand.tsx has visibly irregular JSX indentation, suggesting it wasn't run through a formatter after its last edit.

6. The Design-System Pipeline

The chain: `source/crystal-design-system.html` (4,191 lines, the only place tokens/CSS/icons may be authored) → `tools/extract-design-system.mjs` → four gitignored generated files (`tokens.css`, `crystal.css`, `generated/icons.ts`, `generated/manifest.json`) → the running app. `app.css` is the only hand-authored CSS file (layout composition only), mechanically policed by `.stylelintrc.json` (no literal colours outside tokens). Process is governed by `source/AGENT-PROMPT-apply-design-system.md`'s 14 inviolable rules.

Two genuine, previously-undetected bugs — found by comparing source HTML against generated output

- **A duplicated "CONTAINER GAUGE" section silently loses its first definition.** The source HTML declares this banner section twice (two different, incompatible implementations — different selector scoping, different gap tokens, and a `.cgauge .cg-grille/.cgauge .cg-unit` pair that exists only in the discarded copy). The extractor builds its section lookup as `new Map(sections.map(s => [s.name, s.css]))` — a plain JS Map, so the later duplicate silently wins with no warning. Verified: `crystal.css` contains only the second definition; `generated/manifest.json`'s unfiltered `sections` list is the only artifact that reveals "CONTAINER GAUGE" appears twice.
- **The reduce-motion cherry-pick can never match, by a comma-splitting bug.** The extractor's CHERRY_PICK allow-list stores the manual reduce-motion escape hatch as one three-selector, comma-joined string; `isCherryPicked()` splits the rule's own selector list on commas and checks each piece against the allow-list with `.includes()` — so it is comparing one split piece against the whole joined string, which can never be equal. Verified two ways: `crystal.css` contains zero `.reduce-motion` selectors, and `manifest.json`'s own `droppedSelectors` list names the exact rule that also appears in `cherryPicked` in the same file — the tool's own output visibly contradicts its own config. Currently low-impact (the app only relies on the OS-level prefers-reduced-motion query, not a manual toggle), but a manual reduce-motion switch would silently ship with no CSS behind it.
- Icon dedup uses the **opposite** tie-break rule from CSS-section dedup — icons keep the first occurrence of a duplicate id, sections keep the last. Same file, two contradictory silent tie-breaking policies for the same class of authoring mistake.
- **An unsigned-off contrast correction has been live for 5+ weeks.** `tokens.css`'s source carries a "CONTRAST CORRECTION — 2026-08-11, awaiting sign-off" comment describing seven token-lightness changes made to hit WCAG 4.5:1 — relative to today (2026-09-23), that's over a month of unapproved palette changes live in the shipped app, with no ticket reference anywhere.

- A dead token — `--navy-lift: #6E86D6` — is declared but never referenced; the dark theme's actual accent (`#8DA2E8`) was hardcoded directly instead.

7. End-to-End Test Suite (Playwright)

`playwright.config.ts` runs fully serial (`workers: 1`) against a real running API+web stack, reseeded once via `e2e/globalSetup.ts`. Auth is handled once by `e2e/auth.setup.ts` (signs in as manager, persists `storageState`). `e2e/helpers.ts` centralizes fragile selectors and flows. Five spec files: `interactions.spec.ts` (functional/RBAC), `a11y.spec.ts` (axe-core across 8 screens × 2 themes), `responsive.spec.ts` (phone/tablet/200%-zoom), `screens.spec.ts` (design-system contract assertions, including a genuinely rigorous SVG-geometry check on the readiness gauge), `screenshots.spec.ts` (pure visual capture, zero assertions).

The suite is unusually literate — many assertions cite specific prior defects by name (“the concept used 7.8px fonts to force ten columns onto one screen,” “non-continuous readiness gauges”), effectively serving as institutional memory.

- Several tests hardcode literal computed RGB values tied to design tokens (`interactions.spec.ts` L128: `rgb(11, 19, 48)`; `screens.spec.ts` L163–169: `FFFFFF/rgb(255, 255, 255)/rgb(234, 240, 251)`) instead of comparing self-referentially against the token — any deliberate, correct palette change will fail these for cosmetic reasons.
- `responsive.spec.ts`'s “phone” block sets its own viewport rather than using the dedicated Playwright “phone” project, and has no project restriction — so it silently runs twice per full suite (once under “desktop,” once under “phone”), doubling its runtime for no added coverage.
- Manual fixture-restoration pattern in `interactions.spec.ts` (toggle a checkbox, then toggle it back at the end; edit a note, then clear it back) is fragile — if the test fails partway through, the “undo” step never runs and the shared SQLite fixture is left dirtied for every subsequent test in that run.
- `helpers.ts`'s `openContainer()` always clicks `.first()` of search results with no uniqueness assertion.
- `screenshots.spec.ts` contains zero `expect` calls — it always “passes” regardless of what the images show; visual regressions can only be caught by a human diffing PNGs.

8. Tooling and Workspace Config

`package.json` (root scripts: `setup/dev/build/test/verify`), `tsconfig.base.json` (unusually strict — `noUncheckedIndexedAccess`, `verbatimModuleSyntax`, hard-error `noUnusedLocals/Params`, `forceConsistentCasingInFileNames`, notable given development happens on case-insensitive Windows but presumably deploys to case-sensitive Linux), `eslint.config.js` (flat config, web-only override layering `react-hooks` + `jsx-a11y` with project-specific Radix-aware tweaks), plus per-package manifests for `apps/api`, `apps/web` and `packages/shared`.

- **Possible build-path mismatch:** `apps/api/tsconfig.json` sets `rootDir: "."` while including both `src/**` and `prisma/**`, which should make `tsc -b` emit `dist/src/server.js` — but `apps/api/package.json`'s “start” script runs `node dist/server.js`. Not empirically confirmed (no `node_modules/generated` Prisma client in the sandbox to actually build), but worth verifying with a real `npm run build -w @pms/api && npm start`.

- `packages/shared/package.json`'s exports points directly at `./src/index.ts`, not `dist/` — so the package's own build script's output isn't actually consumed by any resolution path, only serving `tsc -b`'s project-reference bookkeeping.
- `eslint.config.js` globally ignores `apps/api/prisma/**`, so `seed.ts` — a file that ships and executes in production via `db:seed` — is exempt from the `--max-warnings 0` lint gate everything else is held to.
- No Prettier or other formatter config anywhere — style consistency relies entirely on ESLint/stylelint.

9. Auth/Permissions Model (Cross-Cutting)

Four roles, six permissions (`container:read/create/update/advance/task/delete`). Compared directly, the server table (`auth.ts` L50–68) and client table (`lib/auth.tsx` L18–36) are **currently identical** — manager gets all six, supervisor everything but delete, technician read+task, viewer read-only — but they are two independently hand-maintained copies with **no shared source enforcing sync**; `@pms/shared` does not export a permissions table. A server-side policy change requires remembering to hand-edit the client too, and nothing would fail a build if it drifted.

Enforcement is clean on the server (composable `authenticate` + `requirePermission` preHandlers on every mutating route) and explicitly UX-only on the client (`lib/auth.tsx`'s own comment: “the server is still the authority”).

Notably, `supervisor` is a fully-specified role with zero coverage anywhere: not in `api.test.ts`'s `beforeAll`, not in `Login.tsx`'s demo credentials, not in `auth.setup.ts`'s Playwright session. It is the one role nobody — test or human — ever actually exercises.

10. Consolidated Cross-Cutting Risks, Ranked

#	Risk	Where
1	Audit trail is not durable across deletion — ContainerEvent cascades away with its Container on delete; only a log line survives. Directly undercuts the product's stated "audit trail" feature for the one action that matters most.	<code>routes/containers.ts</code> DELETE
2	Login timing side-channel contradicts its own anti-enumeration comment — non-existent emails skip the slow hash entirely, leaking account existence via latency; untested.	<code>routes/auth.ts</code> L17-22
3	"No value typed twice" is violated three separate times outside the design-system pipeline: <code>seed.ts</code> 's hardcoded <code>STAGE_ORDER</code> , <code>readiness.ts</code> 's duplicate <code>totalPlanDaysFor</code> formula, and the hand-duplicated client/server permission tables. Each pair is correct today; none has any protection against silently drifting apart.	<code>seed.ts</code> , <code>readiness.ts</code> , <code>auth.tsx</code>
4	Checklist task-key stability trap — positional keys plus fleet-wide key-based aggregation (<code>averageHoursFor</code>) means an in-the-middle insert/reorder of <code>RAW_TASKS</code> would silently corrupt cross-container averages, with no test or lint catching it.	<code>process.ts</code> , <code>readiness.ts</code>
5	Validation/documentation gaps in <code>schemas.ts</code> — <code>containerId</code> doesn't actually enforce the ISO 6346 shape its comment describes; date fields accept calendar-invalid values shape-only.	<code>schemas.ts</code> L17-31
6	DB has zero enum enforcement — every "enum" is a plain String column; <code>mappers.ts</code> casts DB values to shared union types with unchecked <code>as</code> , and does an <code>un-try/catched JSON.parse</code> on <code>onlyFor</code> .	<code>schema.prisma</code> , <code>mappers.ts</code>
7	Supervisor role has no test or UI coverage anywhere in the stack.	<code>api.test.ts</code> , <code>Login.tsx</code> , <code>auth.setup.ts</code>
8	A placeholder <code>JWT_SECRET</code> is long enough to pass validation — nothing rejects the well-known default at boot.	<code>.env.example</code> , <code>env.ts</code>
9	Design-system pipeline has two silent bugs: the duplicated CONTAINER GAUGE section (last-write-wins Map collision) and the never-matching reduce-motion cherry-pick (comma-split logic error).	<code>extract-design-system.mjs</code>
10	<code>formatDateTime</code> breaks its own module's stated timezone invariant — exactly where it matters most (audit-trail timestamps).	<code>lib/format.ts</code> L25-30
11	An unsigned-off contrast-correction change has been live in production tokens for 5+ weeks with no ticket trail.	<code>tokens.css</code>
12	Two small concrete component bugs: <code>Overlay.tsx</code> 's dead ternary, and <code>RichText.tsx</code> 's premature "saved" state on a failed async save.	<code>Overlay.tsx</code> L39, <code>RichText.tsx</code> L179-182

11. Overall Code-Quality Assessment

Strong

The codebase is unusually self-aware — nearly every file justifies a non-obvious choice inline (env-precedence handling, scrypt-over-bcrypt, the transactional task-toggle, the deliberate absence of a manifest timestamp). The `packages/shared` boundary is genuinely disciplined where it counts: `readiness.ts` is 100% pure, today-threaded functions, and `containers.ts` never reimplements a calculation it could import. The TypeScript baseline is strict well beyond typical projects this size. The design-system extractor, despite its two bugs, is well-engineered (content-hash stamping, a real CSS-aware nested-at-rule-aware splitter, fail-fast guards). Accessibility is a first-class constraint, not an afterthought — enforced structurally (status objects that can't be colour-only, required tooltip labels, `role="alertdialog"` for destructive confirms) rather than just documented. The E2E suite's habit of citing specific historical regressions by name is a genuinely good pattern.

Weak

The recurring pattern across almost every subsystem is *<i>documented intent the code doesn't fully deliver on</i>* — not sloppiness, but comments written to the intended design that the implementation later diverged from (formatDateTime vs. its own module doc; containerId's schema comment vs. its actual regex; the extractor's CHERRY_PICK entry that's listed as wanted but structurally can't match; overview.ts's bandwidth rationale vs. its actual payload). Test coverage has specific, real gaps rather than being broadly thin — no supervisor coverage anywhere, a stateful/order-dependent integration test file, a screenshot spec with zero assertions that always “passes.” Security posture is fine for an internal/demo tool but stacks several unenforced trust boundaries on the same login path (unrejected placeholder secret, timing side-channel, one shared demo password documented in three separate places). Architecturally, the biggest weakness is that the several small “single source of truth” violations found have zero build-time or test-time protection against drifting apart — precisely the failure mode the rest of the architecture was built to prevent everywhere else.

<i>End of report. Produced by a 10-agent parallel deep read of every file in the repository plus a cross-cutting synthesis pass.</i>